第8章　目的コード生成

3番地コードとして生成された命令列は3番地コードを解釈する実行系で実行可能である．3番地コードは計算機のアーキテクチャからは独立した命令であり，変数はメモリ上にあるとして実行される．コンパイラにおける目的コード生成は，中間コードに対応する目的計算機の命令語を選択する．ここで，現在の計算機の多くのアーキテクチャでは，メモリにあるデータを演算器のレジスタにロードして演算を行い，レジスタに得られた結果をメモリに書き戻す動作をする．演算器のレジスタとメモリの間のデータの交換速度は一般に計算機の演算に比べて遅いため，プログラムの解析によってレジスタとメモリの間のデータ交換ができるだけ少なくなるような目的コードを生成することが望ましい．中間コードの変数の数に比べてレジスタの数は非常に少ないため，実行プログラムにおいてその時点で頻繁に演算に用いられる変数をレジスタに置くようなコードを生成することによって実行速度の大幅な改善が可能になる．変数とレジスタの対応を決定することをレジスタ割当という．

一般にプログラムに対して最適なレジスタ割当を行うことは難しく，3番地コードのフローグラフが与えられている時に割当アルゴリズムがNP完全な問題である[?]ため，目的コード生成でのレジスタ割当は最適性を保証することは困難である．従って，目的コード生成では，部分的なプログラムに対して必要なレジスタ数を見積もり，近似的に効率のよい割当手法を与える．具体的には，3番地コードやSSA形式における変数のデータフローからライフサイクル（生存期間）を解析して，変数からレジスタへの割当を決定する．

目的コードの命令の選択については目的マシンのアーキテクチャに応じてできるだけサイクル数の少ない命令（軽い命令）を選択するために命令のコストを見積もるなどの手法がある．しかし，実際の計算機ではハードウェアによる並列的な実行などの効果や実行ユニットの数などに依存することから，実際的な計算機に対するコード生成の系統的なモデル化は教科書的内容の範囲を越えるため，本書では扱わない．

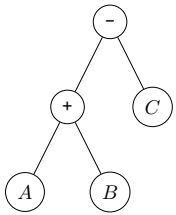
8.1　算術式のコード生成

ここでは，算術式に対するコード生成において必要なレジスタの数を見積もる手法を示す．簡単化のため，以下の範囲に限定する．

- 演算子は四則演算 $(+, -, *, /)$ に限定する．
- 命令はメモリ転送命令 $(\mathtt{Load}\ r, e)$ と演算命令 $(\theta\ r, e)$ とする．
  メモリ転送命令 $\mathtt{Load}\ r, e$ は，e で示されるメモリの内容をレジスタ r に転送する．演算命令 $\theta\ r, e$ は，e で示されるメモリ（またはレジスタ）の内容をフェッチして r と $\theta \in \{+, -, *, /\}$ で定められた演算を行い，結果を r に格納する．

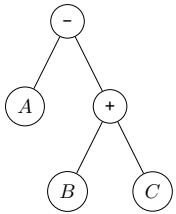
例えば，算術式 $A+B-C$ のコードは以下のようになる．

```
Load  r1  A
+     r1  B
-     r1  C
```



以上の制約のもとでは，レジスタは 1 個で計算が可能である．これに対して，$A-(B+C)$ の場合は以下のような命令系列になる．

```
Load  r2  B
+     r2  C
Load  r1  A
-     r1  r2
```



この算術式の計算に必要なレジスタ数は 2 である．$A*(B+C)$ については，

```
Load  r2  B
+     r2  C
*     r2  A
```

とすればよいので必要なレジスタは 1 つである．これは，$-$ が可換でなく，$*$ が可換であるため 2 番めの項をまず評価して 1 番目の項を適用することができるためである．算術式を表す抽象構文木 T に対して，その計算に必要なレジスタ数を $RN(T)$ と書く．$RN(T)$ はアルゴリズム**??**で求められる．

8.2　グラフ彩色によるレジスタ割当

8.2.1　SSA に対するレジスタ割当

Algorithm 1 $RN(T)$ の計算

```
procedure RN(T)
    if isleaf(T) then return 0                                  ▷ T が葉
    else
        T_1 = Left(T)                                           ▷ T_1 は左部分木
        T_2 = Right(T)                                          ▷ T_2 は右部分木
        if RN(T_1) = 0 かつ RN(T_2) = 0 then return 1           ▷ 左右とも葉なのでレジスタ1つ
        else
            if RN(T_1) = 0 かつ RN(T_2) > 0 then
                if root(T) が可換演算子 then return RN(T_2)     ▷ 後ろから適用する
                else return max(2, RN(T_2))                     ▷ 最大2つ必要
                end if
            else
                if RN(T_1) > 0 then
                    if RN(T_1) = RN(T_2) then return RN(T_1) + 1
                    else return max(RN(T_1), RN(T_2))
                    end if
                end if
            end if
        end if
    end if
end procedure
```